

GROUP PROJECT TECHNICAL REPORT

AI Learning Platform with Intelligent Tutor (Microservices-Based Architecture)

Course: IT41073 - Service Oriented Computing

Project Title: IntelliLearn — Microservices LMS & RAG AI Tutor Platform

Author / Contributor: Chamod (100% solo author contribution across all services)

Date: August 11, 2026

1. Executive Summary & Project Introduction

Modern online education requires scalable, resilient, and intelligent software platforms. The **IntelliLearn Platform** is a cloud-native Learning Management System (LMS) designed using a **Microservices Architecture** paired with an **AI-powered Intelligent Tutor** based on **Retrieval-Augmented Generation (RAG)**.

Traditional monolithic LMS architectures suffer from single points of failure, scaling bottlenecks during exam peaks, and tightly coupled database schemas. In contrast, IntelliLearn decouples business logic into independent microservices (user-service, course-service, quiz-service, progress-service, tutor-service) fronted by a centralized **API Gateway** on port 8080.

Students can enroll in courses, attempt practice assessments with automatic score feedback, track module progress percentages, and query an AI Tutor grounded directly in uploaded course documents (Java_OOP_Guide.pdf, Spring_Boot_Guide.pdf, etc.) to prevent AI hallucinations. Instructors can author courses and prepare assessments through a unified dashboard interface.

2. System Architecture & High-Level Design

2.1 Microservices Architecture Diagram

```
graph TD
    Client["React Frontend Client\n(Port 3000 / Nginx)"]
    
    subgraph Edge_Layer [Edge Layer]
        Gateway["API Gateway\n(Port 8080)\n• Centralized Routing\n• Keycloak OAuth2 / JWT Validation\n• Redis IP Rate Limiting\n• Global CORS Policy"]
    end
    
    subgraph Microservices_Layer [Microservices Layer]
        UserService["User Management Service\n(Port 8081)\n• User Auth & Profiles\n• MongoDB userdb"]
        CourseService["Course Management Service\n(Port 8082)\n• Course Syllabus & Enrolment\n• MongoDB coursedb"]
        QuizService["Quiz & Assessment Service\n(Port 8083)\n• Quiz Player &"]
    end
```

```

Evaluation\n• MongoDB quizdb (quiz_attempts)"]
    ProgressService["Learning Progress Service\n(Port 8084)\n• Progress
Analytics & Percentages\n• MongoDB progressdb"]
    TutorService["AI Tutor Service (RAG + LLM)\n(Port 8085)\n• Course Document
Vector Index\n• Grounded Q&A & Recommendations\n• MongoDB tutordb (interactions)"]
end

subgraph Shared Infrastructure Layer
    MongoDB[(MongoDB Database 7.0\nPersistent Volume: mongodb-data)]
    Redis[(Redis Cache 7.0\nRate Limiting Key-Value)]
    Keycloak[(Keycloak IAM 25.0\nOAuth2 & JWT Issuer)]
end

Client -->|HTTPS / REST API| Gateway
Gateway -->|Redis Rate Limiter| Redis
Gateway -->|Validate JWT Token| Keycloak
Gateway -->|Route /api/users, /api/auth| UserService
Gateway -->|Route /api/courses| CourseService
Gateway -->|Route /api/quizzes| QuizService
Gateway -->|Route /api/progress| ProgressService
Gateway -->|Route /api/tutor| TutorService

UserService --> MongoDB
CourseService --> MongoDB
QuizService --> MongoDB
ProgressService --> MongoDB
TutorService --> MongoDB

```

3. Microservices Detailed Specifications

3.1 User Management Service (user - service)

- **Port:** 8081
- **Database:** MongoDB (userdb, collection users)
- **Security:** Secured with API key user-service-secret-key-123 via ApiKeyAuthFilter.
- **Key Endpoints:**
 - POST /api/auth/register: Creates new student/instructor accounts.
 - POST /api/auth/login: Authenticates credentials and returns JWT bearer token.
 - GET /api/users/{id}: Retrieves profile details.

3.2 Course Management Service (course - service)

- **Port:** 8082
- **Database:** MongoDB (coursedb, collection courses)
- **Security:** Secured with API key course-service-secret-key-456.

- **Key Endpoints:**
 - GET /api/courses: Returns list of all active courses.
 - GET /api/courses/{id}: Returns course syllabus and enrolled student list.
 - POST /api/courses/{id}/enroll: Enrolls student in course.
 - POST /api/courses: Allows instructors (X-User-Role: INSTRUCTOR) to publish new courses.

3.3 Quiz & Assessment Service (quiz-service)

- **Port:** 8083
- **Database:** MongoDB (quizdb, collection quiz_attempts)
- **Security:** Secured with API key quiz-service-secret-key-101.
- **Key Endpoints:**
 - GET /api/quizzes: Returns all available quizzes.
 - POST /api/quizzes/{id}/submit: Evaluates quiz submission, computes score percentage, and persists attempt record to MongoDB (quiz_attempts).
 - GET /api/quizzes/{quizId}/attempts/{userId}: Fetches student quiz attempt history from MongoDB.
 - POST /api/quizzes: Allows admins/instructors (X-User-Role: ADMIN) to create new quizzes.

3.4 Learning Progress Service (progress-service)

- **Port:** 8084
- **Database:** MongoDB (progressdb, collection progress)
- **Security:** Secured with API key progress-service-secret-key-789.
- **Key Endpoints:**
 - GET /api/progress: Fetches all progress records.
 - GET /api/progress/{userId}: Retrieves course completion percentages for a given student.
 - POST /api/progress: Initializes a progress record for course enrollment.
 - PUT /api/progress/{userId}/{courseId}: Updates completion percentage (0-100%).

3.5 AI Tutor Service (RAG + LLM) (tutor-service)

- **Port:** 8085
 - **Database:** MongoDB (tutordb, collection tutor_interactions)
 - **Security:** Secured with API key tutor-service-secret-key-999.
 - **Key Endpoints:**
 - POST /api/tutor/ask: Queries the course document index (Java_OOP_Guide.pdf, Spring_Boot_Guide.pdf, Microservices_Architecture_Spec.pdf). Returns answer, grounded: true/false, and PDF sources.
 - POST /api/tutor/summarize: Generates structured topic summaries.
 - POST /api/tutor/recommend: Returns personalized learning recommendations.
 - GET /api/tutor/health: RAG engine health status.
-

4. Security, API Gateway & Container Infrastructure

4.1 Centralized API Gateway Features

1. **Centralized Routing:** Maps external requests (/api/users/**, /api/courses/**, /api/quizzes/**, /api/progress/**, /api/tutor/**) to internal microservice containers.
2. **Keycloak OAuth2 / JWT Security:** Integrates with Keycloak IAM on port 8180 for token signature verification.
3. **Redis Rate Limiting:** Employs an ipKeyResolver backed by Redis (redis:6379) enforcing replenishRate: 5 requests/sec and burstCapacity: 10 requests/sec to prevent Denial-of-Service (DoS).
4. **Service-Level API Key Injection:** Uses AddRequestHeader filters to attach microservice X-API-KEY tokens to downstream requests automatically.
5. **CORS Governance:** Permits cross-origin calls from http://localhost:3000 with support for GET, POST, PUT, DELETE, OPTIONS.

4.2 Docker Compose Ecosystem (10 Containers)

The entire system deploys via a single command (docker compose up -d --build):

1. api-gateway (Port 8080)
2. client (React + Nginx, Port 3000)

3. user-service (Port 8081)
 4. course-service (Port 8082)
 5. quiz-service (Port 8083)
 6. progress-service (Port 8084)
 7. tutor-service (Port 8085)
 8. mongodb (Mongo 7.0, Port 27017, Volume mongodb-data:/data/db)
 9. redis (Redis 7.0, Port 6379)
 10. keycloak (Keycloak 25.0, Port 8180)
-

5. Empirical Test & Verification Matrix

Verification Test	Command / Method	Expected Result	Status
Gateway Health	curl -i http://localhost:8080/gateway/health	200 OK {"status":"Gateway is running"}	PASS
Quiz Endpoint	curl -i http://localhost:8080/api/quizses	200 OK Array of seeded quizzes	PASS
Quiz Submission	POST http://localhost:8080/api/quizses/1/submit	200 OK {"score":100, "correctAnswersCount":3}	PASS
Quiz Attempt History	GET http://localhost:8080/api/quizses/1/attempts/student1@intellilearn.com	200 OK Record fetched from MongoDB	PASS
MongoDB Persistence	docker compose restart mongodb quiz-service	Attempt record preserved in MongoDB	PASS
AI Tutor RAG Engine	POST http://localhost:8080/api/tutor/ask (Polymorphism)	200 OK "grounded": true, "sources": ["Java_OOP_Guide.pdf"]	PASS

Verification Test	Command / Method	Expected Result	Status
AI Tutor Fallback	POST http://localhost:8080/api/tutor/ask (Chocolate cake query)	200 OK "grounded": false, "sources": []	PASS
AI Recommendations	POST http://localhost:8080/api/tutor/recommend	200 OK Personalized study advice	PASS
CORS Preflight	OPTIONS http://localhost:8080/api/quizzes	200 OK Access-Control-Allow-Origin: http://localhost:3000	PASS
Swagger UI	http://localhost:8080/swagger-ui.html	302 Redirect to interactive Swagger UI on all services	PASS

6. Individual Contribution Matrix

Note: Chamod performed 100% of all microservices implementation, API gateway configuration, MongoDB persistence integration, AI RAG tutor development, Docker orchestration, and React client frontend engineering.

Component / Task	Contribution %	Author
API Gateway & Security Configuration	100%	Chamod
User Service (user-service)	100%	Chamod
Course Service (course-service)	100%	Chamod
Quiz & Assessment Service (quiz-service)	100%	Chamod
Learning Progress Service (progress-service)	100%	Chamod
AI Tutor Service (tutor-service RAG)	100%	Chamod
Docker Compose Orchestration (10 Containers)	100%	Chamod

Component / Task	Contribution %	Author
React Client Frontend & Integration	100%	Chamod